

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΤΜΗΜΑ ΜΗΧ Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ
ΤΟΜΕΑΣ ΥΛΙΚΟΥ ΚΑΙ ΑΡΧΙΤΕΚΤΟΝΙΚΗΣ ΥΠΟΛΟΓΙΣΤΩΝ
Εργαστήριο Ηλεκτρονικών & Λογικού Σχεδιασμού

ΕΡΓΑΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ
ΛΟΓΙΚΟΥ ΣΧΕΔΙΑΣΜΟΥ

Γ.Φ. ΑΛΕΞΙΟΥ - Ε.Ζ. ΨΑΡΑΚΗΣ

Πάτρα 2018

Ακαδ. Έτος : 2020-21

Εαρινό Εξάμηνο : 2020-21

Ονοματεπώνυμο: ΚΙΟΥΡΤ-ΜΕΧΜΕΤΑΛΗ ΑΧΜΕΤ

Α.Μ. : 1084672

Ονομ/νυμο Συνεργάτη:_____

Α.Μ. : _____

Ονομ/νυμο Συνεργάτη:_____

Α.Μ. : _____

Τμήμα : _____

Άσκηση: __ ΑΣΚΗΣΗ 3 __

Αναλυτική Βαθμολογία Πρακτικού Μέρους:

	Βαθμός Αριθ.	Αξιολογητή Ολογραφ.	Υπογραφή Αξιολογητή
<i>Άσκηση 1</i>			
<i>Άσκηση 3</i>			
<i>Άσκηση 5</i>			
<i>Άσκηση 6</i>			
<i>Άσκηση 7</i>			
<i>Μ.Ο.</i>			

ΑΣΚΗΣΗ 3

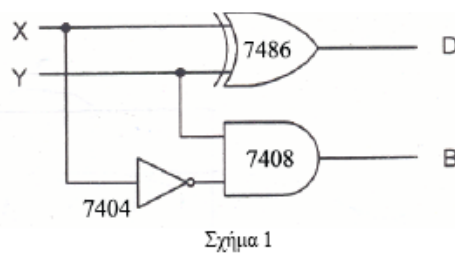
ΗΜΕΡΟΜΗΝΙΑ:

ΘΕΜΑ: ΗΜΙΑΦΑΙΡΕΤΗΣ, ΑΦΑΙΡΕΤΗΣ -

HALF, FULL SUBTRACTOR (Morris Mano, Κεφ. 1- 4).

ΘΕΩΡΙΑ

ΗΜΙΑΦΑΙΡΕΤΗΣ (HALF SUBTRACTOR, HS). Ο HS είναι ένα συνδυαστικό κύκλωμα δύο εισόδων **X**, **Y** και δύο εξόδων **D** (Difference) και **B** (Borrow). Το κύκλωμα αυτό δίνει στην έξοδο του **D** τη διαφορά (**X**- **Y**) των δύο εισόδων και στο **B** μας λέει αν χρειαστήκαμε να δανειστούμε μια μονάδα για την αφαίρεση. Το κύκλωμα και οι λογικές συναρτήσεις των δύο εξόδων ακολουθούν.



$$D = X' Y + X Y' = X \text{ xor } Y$$

$$B = X' Y$$

ΔΙΑΔΙΚΑΣΙΑ

1. Σχεδιάστε στο simulator, ένα συνδυαστικό κύκλωμα που δέχεται ως **είσοδο** ένα προσημασμένο αριθμό των 4-bits σε αναπαράσταση "συμπληρώματος ως προς 2", $X = X_3X_2X_1X_0$ και δίνει ως **έξοδο** την αριθμητική τιμή της εισόδου σε αναπαράσταση "Πρόσημου-Μέτρου" (Sign-Magnitude), $Y = sY_2Y_1Y_0$.

Ισοδύναμα, το κύκλωμα πρέπει να έχει τις ακόλουθες ιδιότητες:

- α. Όταν $X \geq 0$ ($X_3=0$) τότε $Y = X$
- β. Όταν $X < 0$ ($X_3=1$), το Y παριστάνει την Sign-Magnitude τιμή της εισόδου.

Δικαιολογείστε την απάντησή σας (πίνακας αληθείας, απλοποιήσεις, κλπ.) και δώστε τα ζητούμενα screenshots από τον simulator.

Παρατηρήσεις:

- Χρησιμοποιείτε ολοκληρωμένα της επιλογής σας, **της βιβλιοθήκης TTL**. Υπόδειξη: μετά από απλοποίηση το κύκλωμα δύναται να υλοποιηθεί μόνο με 3 ολοκληρωμένα (συνολικά 5 πύλες).
- Συνδέστε τις εισόδους, $X_3X_2X_1X_0$, σε pins εισόδου για να τους αποδώσετε τιμές και τις εξόδους, $sY_2Y_1Y_0$, σε LEDs για να βεβαιωθείτε ότι το κύκλωμά σας δουλεύει σωστά.
- Συνδέστε τις εξόδους **ΚΑΙ** σε pins εξόδου, γιατί θα χρησιμοποιήσετε το κύκλωμα αυτό ως υποκύκλωμα στο Ερώτ. 3.Β.

ΠΙΝΑΚΑΣ ΑΛΗΘΕΙΑΣ

Δεκαδικό	Signed Magnitude	2's complement
+7	0111	0111
+6	0110	0110
+5	0101	0101
+4	0100	0100
+3	0011	0011
+2	0010	0010
+1	0001	0001
0	0000 1000	0000
-1	1001	1111
-2	1010	1110
-3	1011	1101
-4	1100	1100
-5	1101	1011
-6	1110	1010
-7	1111	1001
-8	1000	1000

X3	X2	X1	X0	s	Y2	Y1	Y0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	1	1	1
1	0	1	0	1	1	1	0
1	0	1	1	1	1	0	1
1	1	0	0	1	1	0	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	1	0
1	1	1	1	1	0	0	1

Χάρτης Karnaugh για Y2

x3x2/x1x0	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	0	0	0
10	0	1	1	1

Χάρτης Karnaugh για S

x3x2/x1x0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

Χάρτης Karnaugh για Y0

x3x2/x1x0	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

Χάρτης Karnaugh για Y1

x3x2/x1x0	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	1	0	1
10	0	1	0	1

ΛΟΓΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ΕΞΟΔΩΝ s, Y₂, Y₁, Y₀

$$S = x_3$$

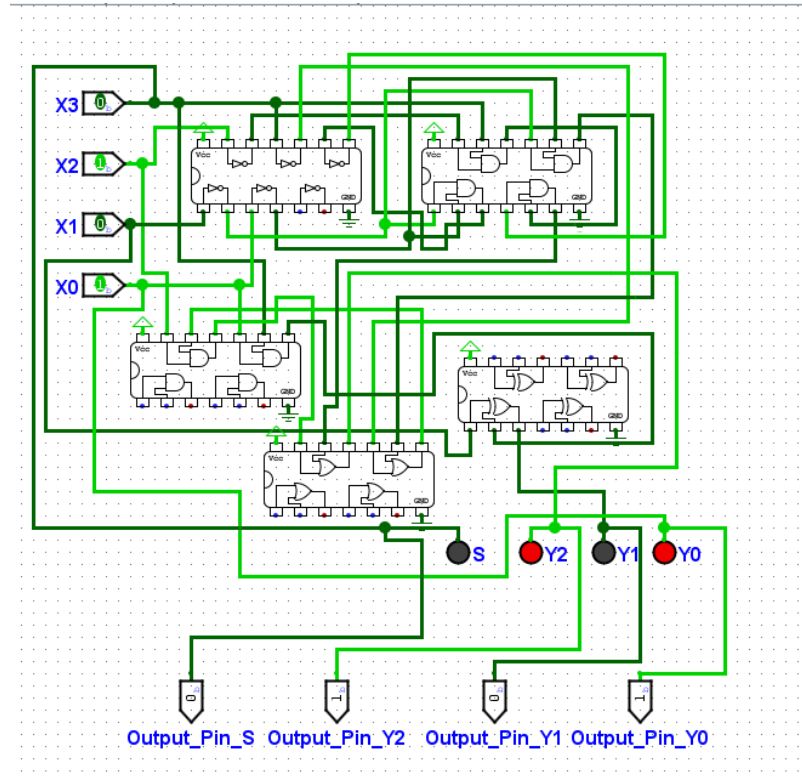
$$Y_2 = x_3'x_2 + x_3x_2'x_0 + x_3x_2'x_1 + x_2x_1'x_0' = x_3'x_2 + x_3x_2'(x_1+x_0) + x_2x_1'x_0' = x_3'x_2 + x_3x_2'(x_1'x_0')' + x_2x_1'x_0' = x_3x_2'(x_1'x_0')' + x_2(x_3' + x_1'x_0')$$

$$Y_1 = x_3'x_1 + x_3x_1'x_0 + x_1x_0' = x_1 \text{ xor } x_3x_0$$

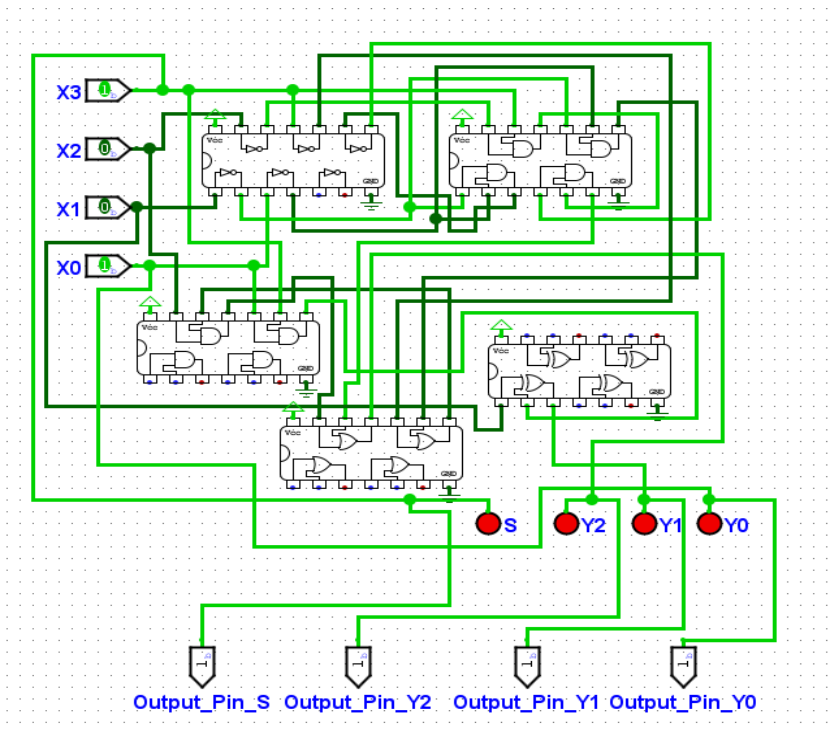
$$Y_0 = x_0$$

ΚΥΚΛΩΜΑ

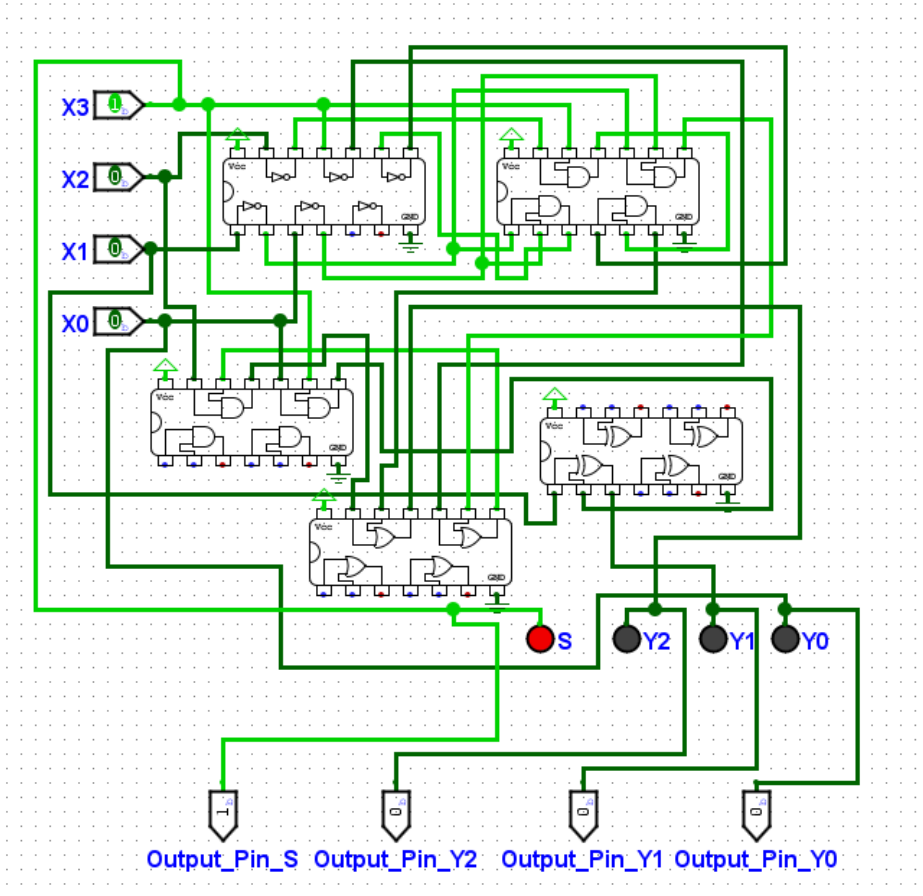
- Screenshot για την περίπτωση $X = 5$



- Screenshot για την περίπτωση $X = -7$



- Screenshot για την περίπτωση $X = -8$



2. Ένας Πλήρης Αφαιρέτης (Full Subtractor, FS) είναι ένα συνδυαστικό κύκλωμα τριών εισόδων **X**, **Y**, **Bin** και δύο εξόδων **B_{out}**, **D**. Η έξοδος **B_{out}** είναι το πιθανό δανεισμένο bit και η έξοδος **D** είναι η διαφορά των τριών εισόδων (**X - Y - Bin**).

α. Γράψτε τον πίνακα αλήθειας του FS και υπολογίστε τις λογικές συναρτήσεις των εξόδων B_{out}, D.

ΠΙΝΑΚΑΣ ΑΛΗΘΕΙΑΣ ΚΑΙ ΛΟΓΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ΓΙΑ B_{out}, D

X	Y	Bin	Bout	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Bout =

X/YBin	00	01	11	10
0	0	1	1	1
1	0	0	1	0

$$\text{Bout} = Y\text{Bin} + X' \text{Bin} + X' Y$$

D =

X/YBin	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$D = X \text{ xor } Y \text{ xor } \text{Bin}$$

β. Σχεδιάστε στον simulator έναν FS, χρησιμοποιώντας ως υποκυκλώματα δύο ημι-αφαιρέτες (Half-Subtractors, HS) και δώστε τα ζητούμενα screenshots από τον simulator.

Παρατηρήσεις για τον HS (δείτε το Σχήμα 1):

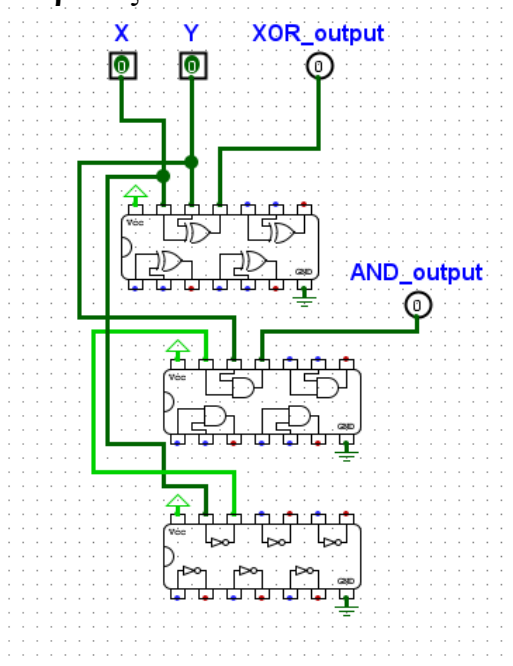
- Χρησιμοποιείτε τα ολοκληρωμένα 7404, 7408, 7486 **από την βιβλιοθήκη TTL**.
- Συνδέστε τις εισόδους X, Y, Bin και τις εξόδους, Bout, D, σε pins εισόδου και εξόδου αντίστοιχα, γιατί θα χρησιμοποιήσετε το κύκλωμα του HS ως υποκύκλωμα για τον FS.
- Συμπεριλάβετε σε ξεχωριστό screenshot το κύκλωμα του HS

Παρατηρήσεις για τον FS:

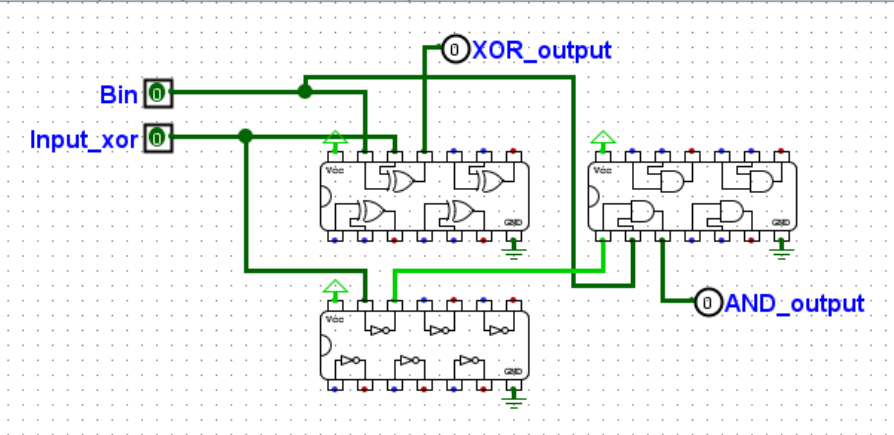
- Χρησιμοποιείτε τον HS ως υποκύκλωμα και επιπλέον ένα ολοκληρωμένο 7432, **από την βιβλιοθήκη TTL**.
- Συνδέστε τις εξόδους σας **και** σε LEDs για να βεβαιωθείτε ότι το κύκλωμά σας δουλεύει σωστά.

ΚΥΚΛΩΜΑ HS (screenshot)

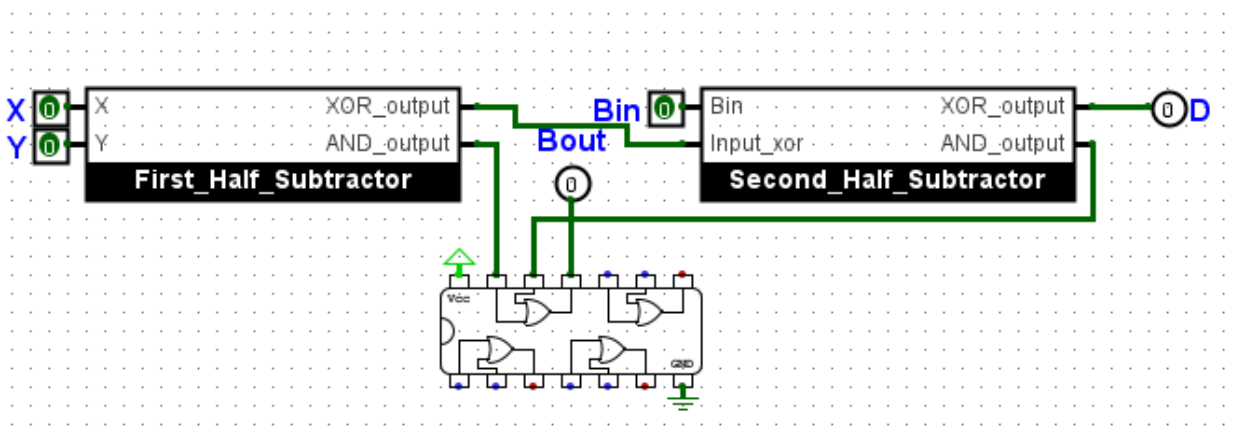
Ο πρώτος HS



Ο δεύτερος HS

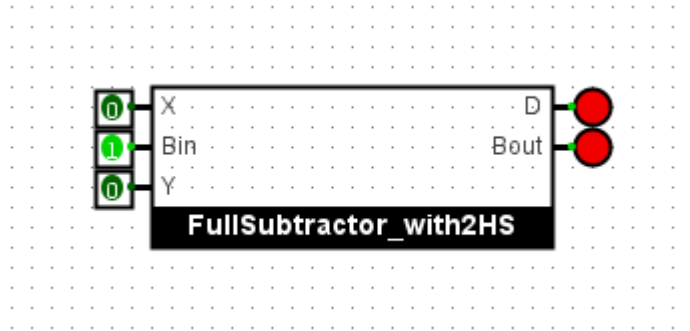


Συνδεμένα σε ένα chip OR για ολοκλήρωση ενός FS

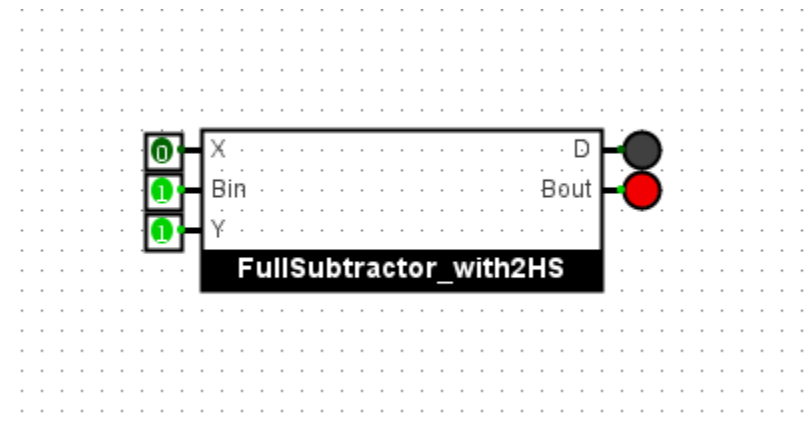


ΚΥΚΛΩΜΑ FS (με 2 HS ως υποκυκλώματα)

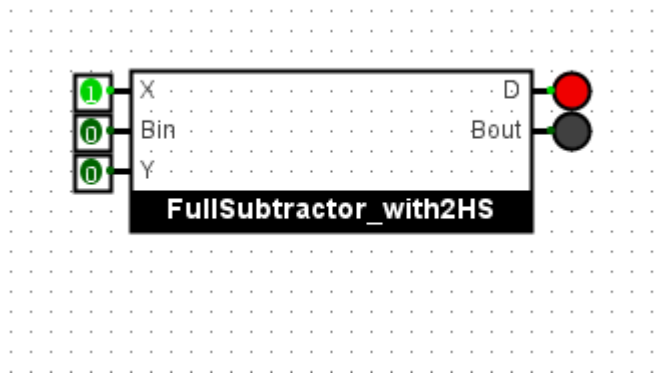
- Screenshot για την περίπτωση $X = 0, Y=0, Bin=1$



- Screenshot για την περίπτωση $X = 0, Y=1, Bin=1$



Screenshot για την περίπτωση $X = 1, Y = 0, Bin = 0$



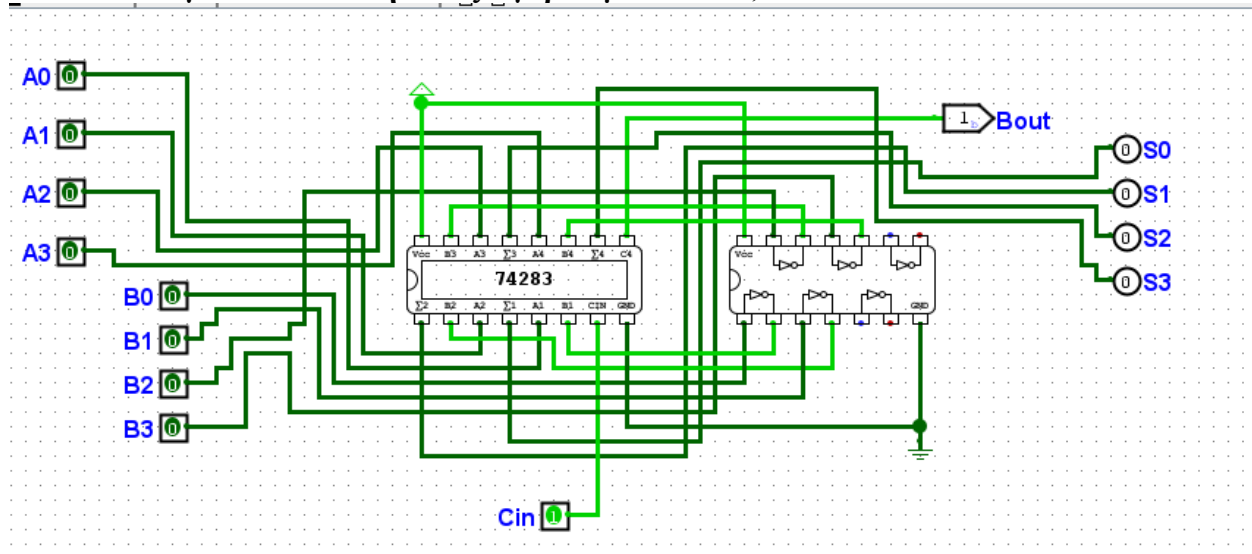
3.A. Σχεδιάστε στον simulator έναν αφαιρέτη των 4-bits, χρησιμοποιώντας από την βιβλιοθήκη TTL: ένα (1) ολοκληρωμένο **74LS283**, (4-bit Full-Adder, αθροιστής, βλ. ΠΑΡΑΡΤΗΜΑ, σελ17), και οποιοδήποτε άλλο ολοκληρωμένο κρίνεται απαραίτητο (πάλι από την βιβλιοθήκη **TTL**) και δώστε τα ζητούμενα screenshots από τον simulator.

Παρατηρήσεις:

- Συνδέστε τις εισόδους του αφαιρέτη σε pins εισόδου (για απόδοση τιμών) και τις εξόδους του σε LEDs για να επιβεβαιώσετε τη σωστή λειτουργία του (αποτέλεσμα σε 2's complement).
- Συνδέστε τις εξόδους του αφαιρέτη **ΚΑΙ** σε pins εξόδου, **για να χρησιμοποιήσετε τον αφαιρέτη ως υποκύκλωμα στο συνολικό κύκλωμα του Ερωτ. 3.B** (Σχήμα 2).

ΚΥΚΛΩΜΑ ΑΦΑΙΡΕΤΗ των 4-bits

- Screenshot για ένα οποιοδήποτε ζευγάρι τιμών των A, B

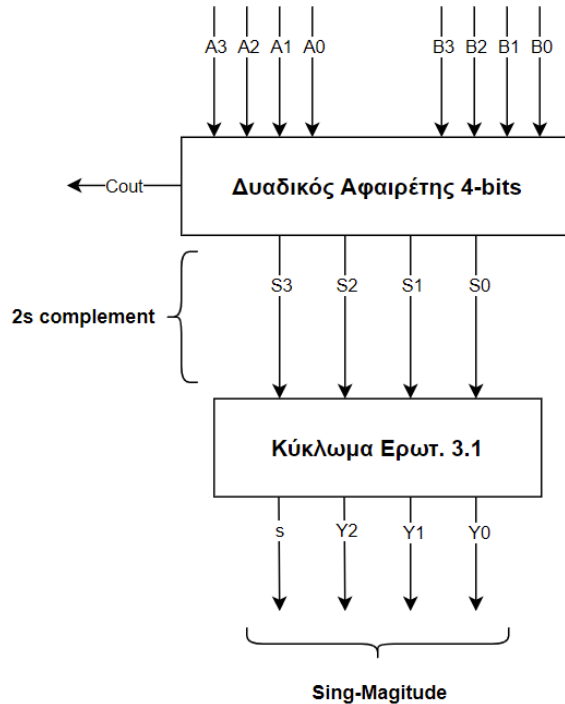


Σύντομη δικαιολόγηση του κυκλώματος που σχεδιάσατε

Για να φτιάξουμε ένα κύκλωμα Full Subtractor για A και B όπως βλέπετε χρειάζεται να συνδέσουμε τα pin εισόδου στο Full Adder (Λόγο της άσκησης). Αλλά για να γίνει ένας Full Subtractor χρειάζεται και είναι απαραίτητο το chip hex inverter ώστε να μετατραπούν οι είσοδοι των B που με αυτό το τρόπο αλλάζει το κύκλωμα μας από Adder σε Subtractor.

3.B. Χρησιμοποιήστε τον αφαιρέτη του Ερωτ. 3.A ως υποκύκλωμα και σχεδιάστε το συνολικό κύκλωμα του Σχήματος 2. Συνδέστε τις εξόδους του αφαιρέτη ($S_3S_2S_1S_0$), αγνοώντας το τελικό κρατούμενο, στο συνδυαστικό κύκλωμα του **ερωτήματος 3.1**, επαληθεύστε ότι παίρνετε τα σωστά αποτελέσματα σε αναπαράσταση “Signed-Magnitude” (Σχήμα 2), και δώστε τα ζητούμενα screenshots από τον simulator.

Κύκλωμα Αφαιρέτη των 4-bits + Κύκλωμα Ερωτ. 3.1.



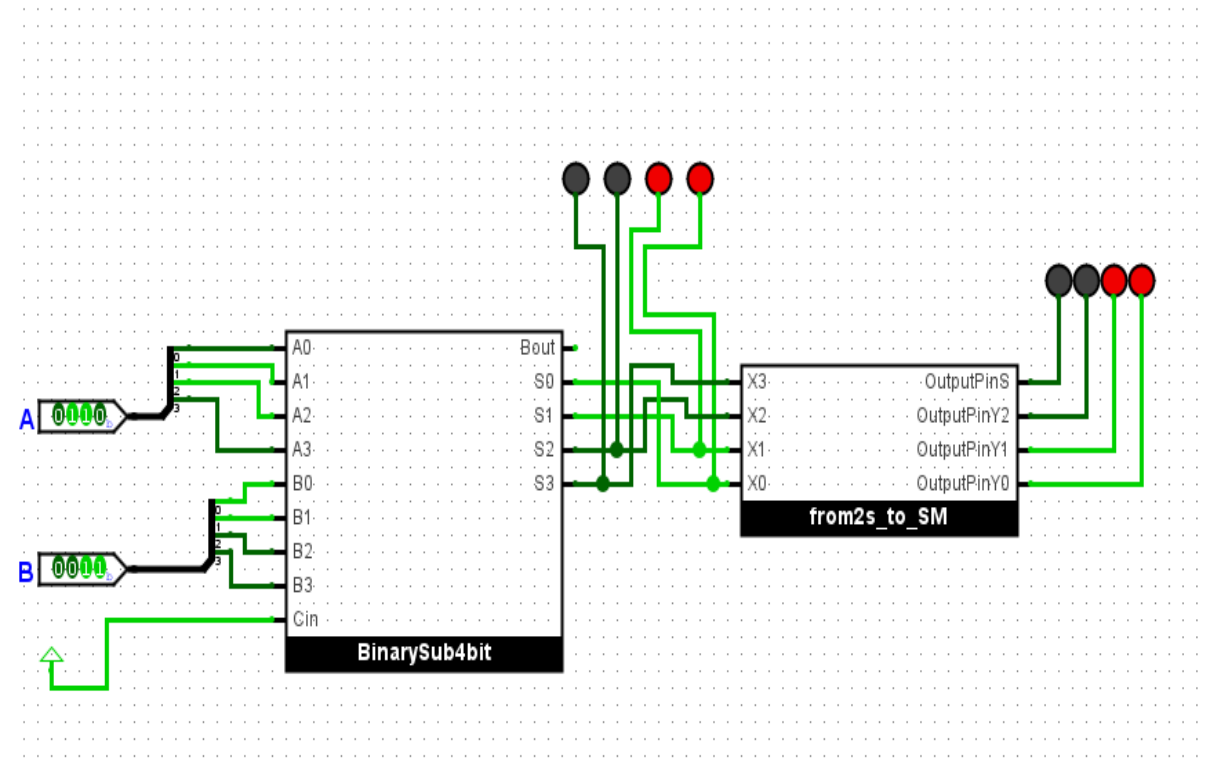
Σχήμα 2

Παρατηρήσεις:

- Συνδέστε τις εισόδους του ολικού κυκλώματος σε pins εισόδου για να αποδώσετε τιμές στα A_i , B_i , ($i = 0 \dots 3$) και τις εξόδους του κυκλώματος του Ερωτ. 3.1 ($sY_2Y_1Y_0$) σε LEDs, για να βεβαιωθείτε ότι το κύκλωμά σας δίνει τα σωστά τελικά αποτελέσματα **σε signed-magnitude αναπαράσταση**.
- Μπορείτε να συνδέσετε και τις εξόδους του αφαιρέτη ($S_3S_2S_1S_0$) σε LEDs ώστε να παρατηρείτε το αποτέλεσμα της αφαίρεσης και σε συμπλήρωμα ως προς 2 (2's complement)

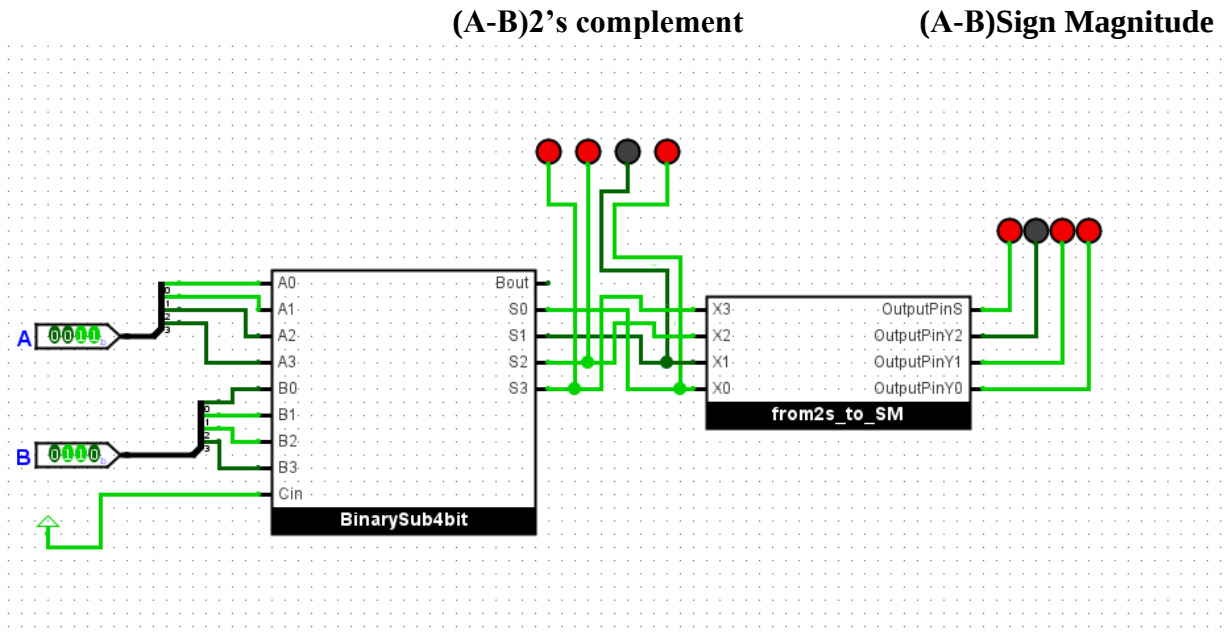
ΚΥΚΛΩΜΑ ΑΦΑΙΡΕΤΗ των 4-bits + ΚΥΚΛΩΜΑ ΕΡΩΤΗΜΑΤΟΣ 3.1 (Σχήμα 2)

- Screenshot για την περίπτωση $A = 6_{10} = 0110_2$, $B = 3_{10} = 0011_2$

**Δικαιολόγηση αποτελέσματος ($A - B = 6 - 3$)**

Στο κύκλωμα μας βλέπουμε ότι το αποτέλεσμα του 2's complement και Sign Magnitude είναι ίδιο που βγάζουμε το συμπέρασμα ότι το αποτέλεσμα μας είναι ένας θετικός αριθμός. Που είναι 3.

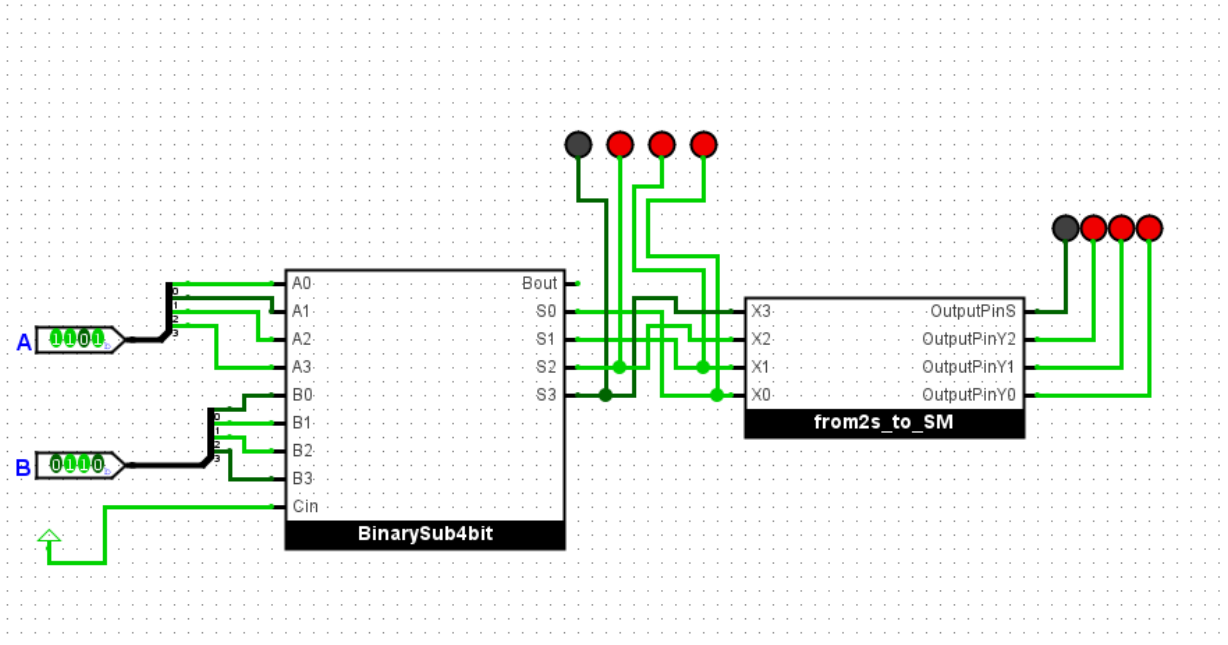
- Screenshot για την περίπτωση περίπτωση $A = 3_{10} = 0011_2$, $B = 6_{10} = 0110_2$



Δικαιολόγηση αποτελέσματος ($A - B = 3 - 6$)

Αυτή την φορά μας βγάζει έναν αρνητικό αριθμό που βλέπουμε ότι είναι διαφορετικά τα 2's complement και Sign Magnitude. Έτσι μπορούμε να καταλάβουμε και καλύτερα ότι δεν υπάρχει πρόβλημα στο κύκλωμα μας.

- Screenshot για την περίπτωση περίπτωση $A = -3_{10} = 1101_2$, $B = 6_{10} = 0110_2$



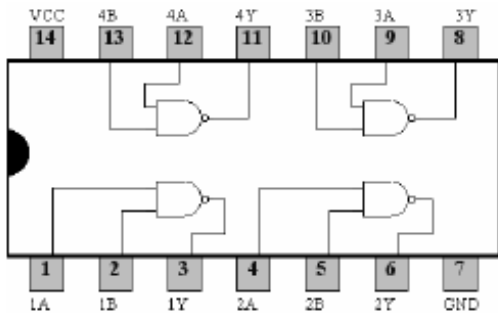
Δικαιολόγηση αποτελέσματος ($A - B = -3 - 6$)

Σε αυτή την περίπτωση του κυκλώματος μας έχουμε το $-3 - 6 = -9$. Επειδή όμως μας τελειώνουν τα bits δεν μπορούμε να το αναπαραστήσουμε το -9. Είτε σε 2s complement είτε σε signed magnitude. Αν λαμβάναμε υπόψη και το Bout του 4bit full subtractor τότε μόνο θα ήταν σωστό το 2s complement.

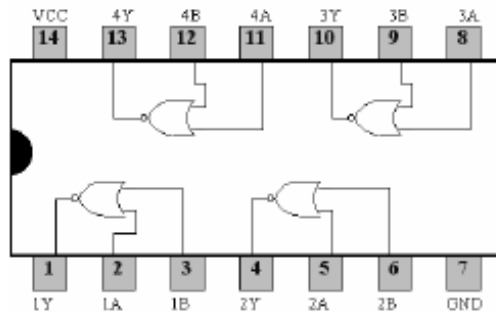
ΒΟΗΘΗΤΙΚΗ ΣΕΛΙΔΑ.

ΠΑΡΑΡΤΗΜΑ

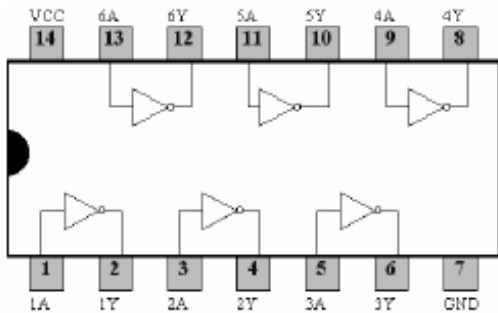
Διαγράμματα Ολοκληρωμένων



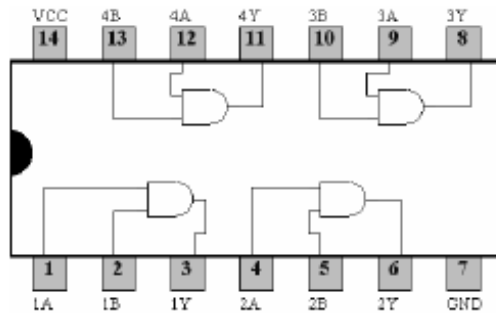
7400 Quad 2-Input NAND Gate



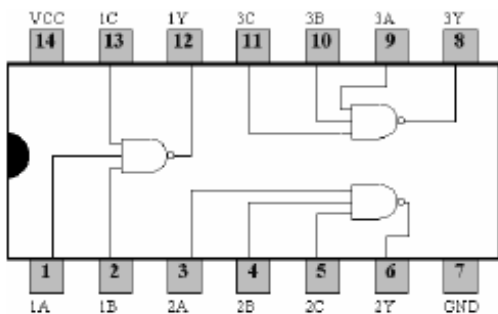
7402 Quad 2-Input NOR Gate



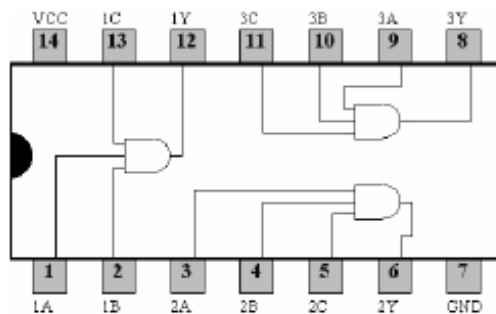
7404 Hex Inverter



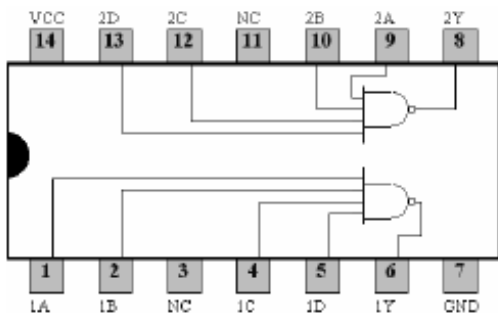
7408 Quad 2-Input AND Gate



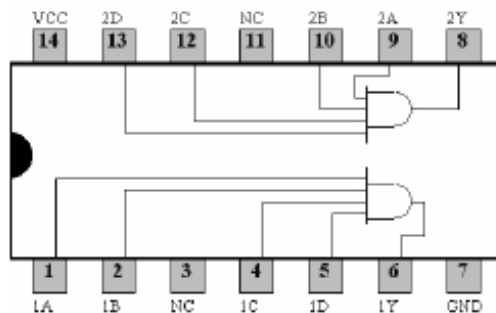
7410 Triple 3-Input NAND Gate



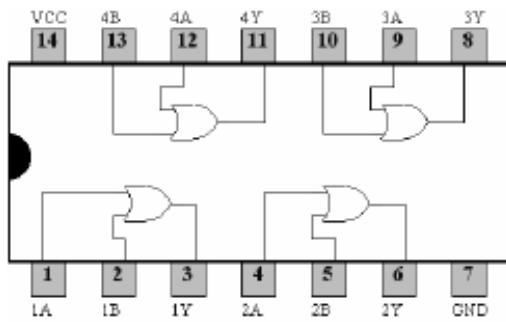
7411 Triple 3-Input AND Gate



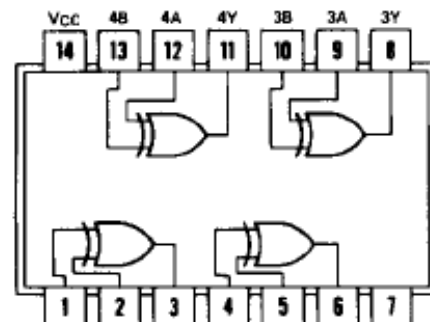
7420 Dual 4-Input NAND Gate



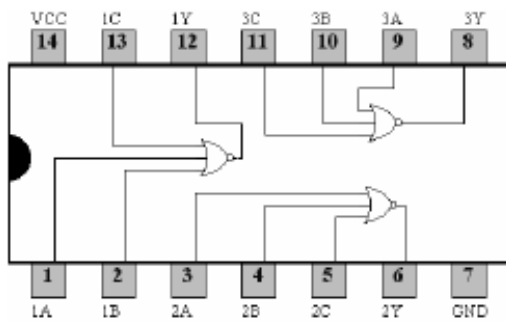
7421 Dual 4-Input AND Gate



7432 Quad 2-Input OR Gate



7486 Quad 2-Input XOR Gate

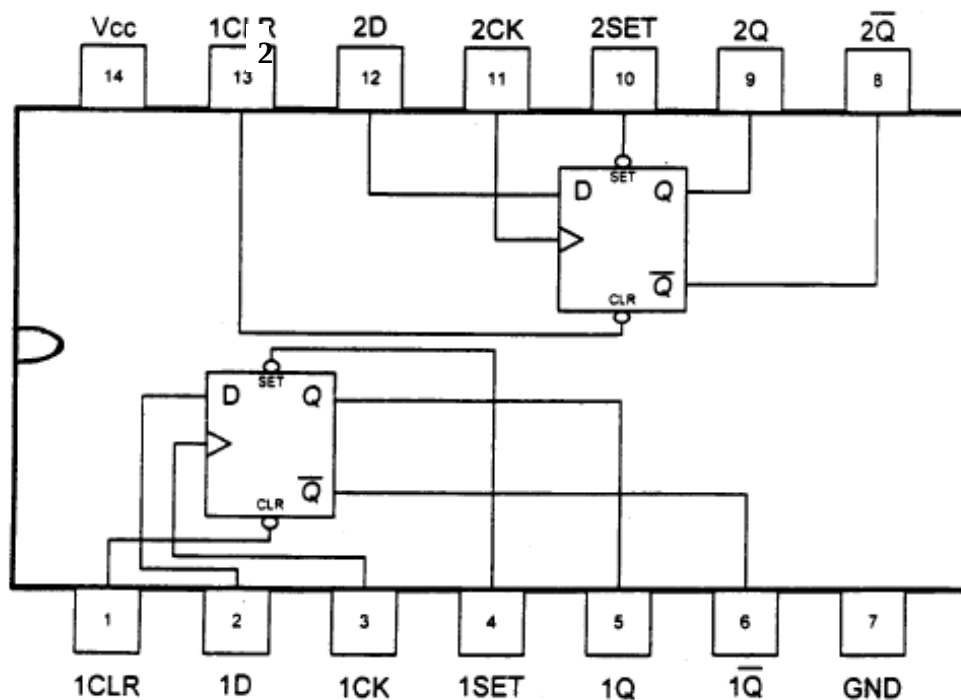


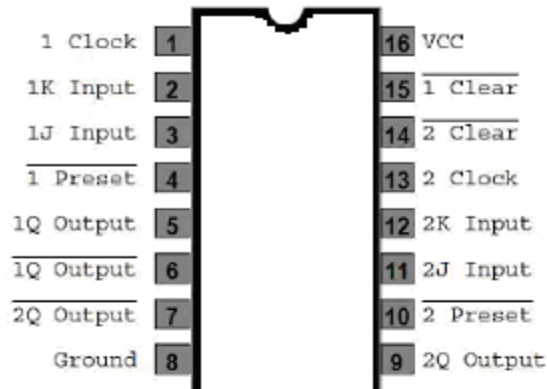
7427 Triple 3-Input NOR Gate

DUAL D-TYPE POSITIVE EDGE TRIGGERED FLIP-FLOPS WITH PRESET AND CLEAR

FUNCTION TABLE

INPUTS				OUTPUTS	
PRESET	CLEAR	CLOCK	D	Q	Q'
L	H	X	X	H	L
H	L	X	X	L	H
L	L	X	X	H	H
H	H	↑	H	H	L
H	H	↑	L	L	H
H	H	L	X	Q ₀	Q' ₀



DUAL JK FLIP_FLOP with CLEAR/PRESET**74112****Function Table**

Inputs					Outputs	
PR	CLR	CLK	J	K	Q	$\overline{Q}$
L	H	X	X	X	H	L
H	L	X	X	X	L	H
L	L	X	X	X	H (Note 1)	H (Note 1)
H	H	↓	L	L	Q_0	$\overline{Q}_0$
H	H	↓	H	L	H	L
H	H	↓	L	H	L	H
H	H	↓	H	H	Toggle	
H	H	H	X	X	Q_0	$\overline{Q}_0$

H = HIGH Logic Level

L = LOW Logic Level

X = Either LOW or HIGH Logic Level

↓ = Negative Going Edge of Pulse

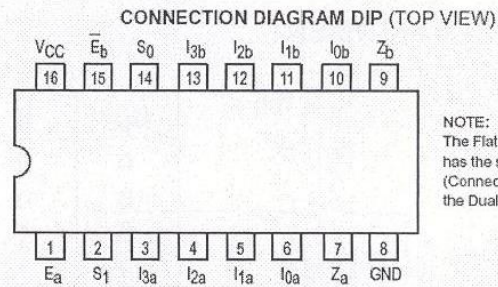
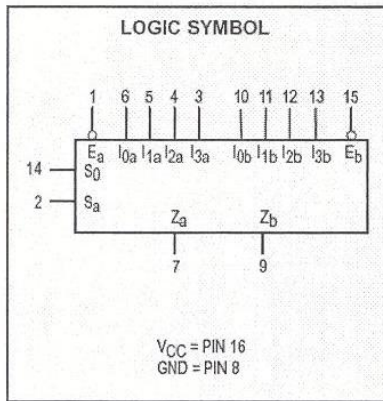
 Q_0 = The output logic level before the indicated input conditions were established.

Toggle = Each output changes to the complement of its previous level on each falling edge of the clock pulse.

Note 1: This configuration is nonstable; that is, it will not persist when preset and/or clear inputs return to their inactive (HIGH) level.

74153

DUAL 4-INPUT MULTIPLEXER



NOTE:
The Flatpak version
has the same pinouts
(Connection Diagram) as
the Dual In-Line Package.

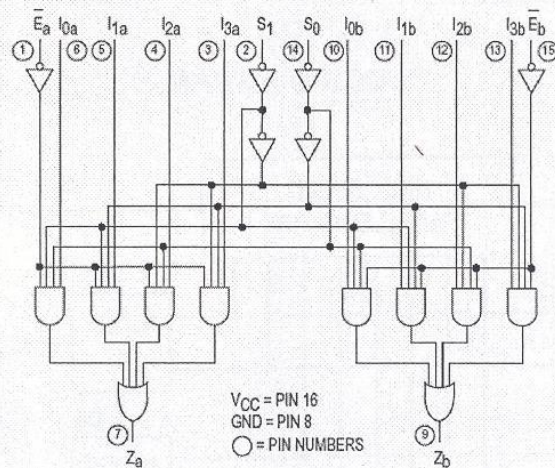
PIN NAMES

S_0 Common Select Input
 E Enable (Active LOW) Input
 I_0, I_1 Multiplexer Inputs
 Z Multiplexer Output (Note b)

LOADING (Note a)

HIGH	LOW
0.5 U.L.	0.25 U.L.
0.5 U.L.	0.25 U.L.
0.5 U.L.	0.25 U.L.
10 U.L.	5 (2.5) U.L.

LOGIC DIAGRAM



TRUTH TABLE

SELECT INPUTS		INPUTS (a or b)						OUTPUT
S_0	S_1	E	I_0	I_1	I_2	I_3		Z
X	X	H	X	X	X	X		L
L	L	L	L	X	X	X		L
L	L	L	H	X	X	X		H
H	L	L	X	L	X	X		L
H	L	L	X	H	X	X		H
L	H	L	X	X	L	X		L
L	H	L	X	X	H	X		H
H	H	L	X	X	X	L		L
H	H	L	X	X	X	H		H

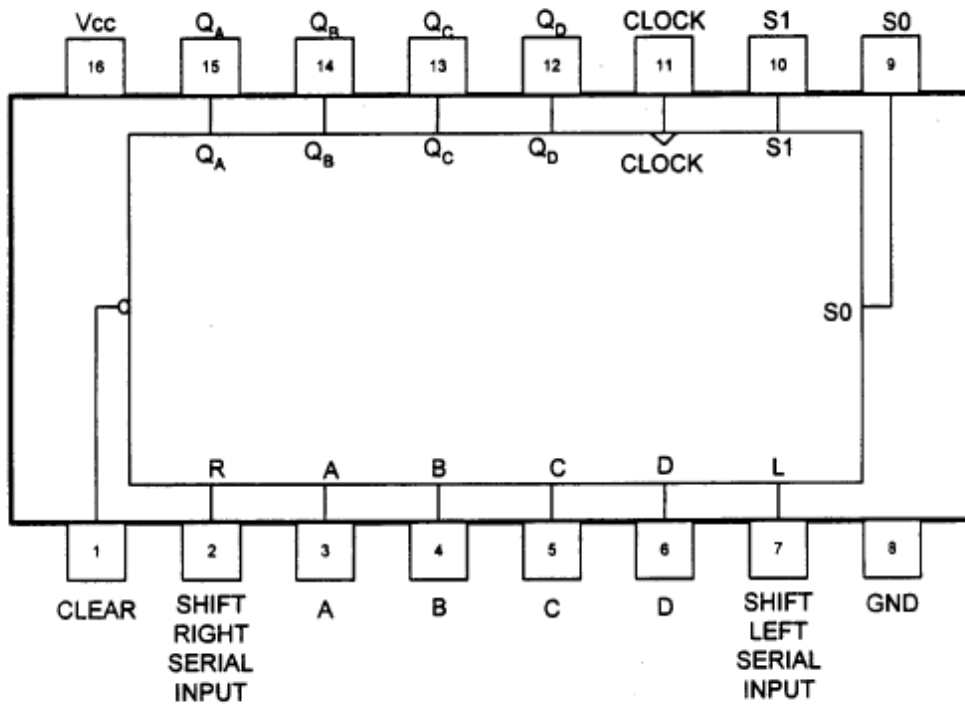
H = HIGH Voltage Level
L = LOW Voltage Level
X = Don't Care

74194

4 - BIT BIDIRECTIONAL UNIVERSAL SHIFT REGISTER

FUNCTION TABLE

INPUTS										OUTPUTS			
CLEAR	MODE		CLOCK	SERIAL		PARALLEL				Q _A	Q _B	Q _C	Q _D
	S ₁	S ₀		LEFT	RIGHT	A	B	C	D				
L	X	X	X	X	X	X	$\bar{E}_a$	X	X	L	L	L	L
H	X	X	L	X	X	X	X	X	X	Q _{A0}	Q _{B0}	Q _{C0}	Q _{D0}
H	H	H	↑	X	X	a	b	c	d	a	b	c	d
H	L	H	↑	X	H	X	X	X	X	H	Q _{An}	Q _{Bn}	Q _{Cn}
H	L	H	↑	X	L	X	X	X	X	L	Q _{An}	Q _{Bn}	Q _{Cn}
H	H	L	↑	H	X	X	X	X	X	Q _{Bn}	Q _{Cn}	Q _{Dn}	H
H	H	L	↑	L	X	X	X	X	X	Q _{Bn}	Q _{Cn}	Q _{Dn}	L
H	L	L	X	X	X	X	X	X	X	Q _{A0}	Q _{B0}	Q _{C0}	Q _{D0}





August 1986
Revised March 2000

DM74LS283

4-Bit Binary Adder with Fast Carry

General Description

These full adders perform the addition of two 4-bit binary numbers. The sum (Σ) outputs are provided for each bit and the resultant carry (C4) is obtained from the fourth bit. These adders feature full internal look ahead across all four bits. This provides the system designer with partial look-ahead performance at the economy and reduced package count of a ripple-carry implementation.

The adder logic, including the carry, is implemented in its true form meaning that the end-around carry can be accomplished without the need for logic or level inversion.

Features

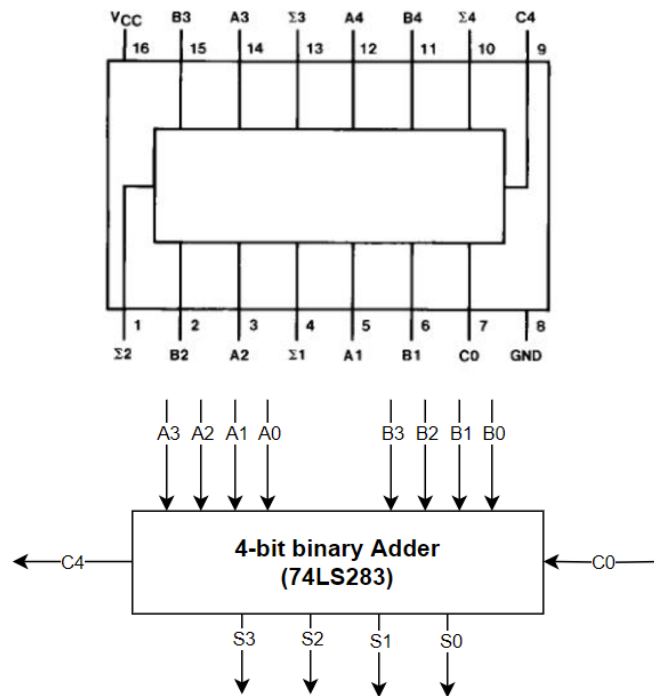
- Full-carry look-ahead across the four bits
- Systems achieve partial look-ahead performance with the economy of ripple carry
- Typical add times
 - Two 8-bit words 25 ns
 - Two 16-bit words 45 ns
- Typical power dissipation per 4-bit adder 95 mW

Ordering Code:

Order Number	Package Number	Package Description
DM74LS283M	M16A	16-Lead Small Outline Integrated Circuit (SOIC), J EDEC MS-012, 0.150 Narrow
DM74LS283N	N16E	16-Lead Plastic Dual-In-Line Package (PDIP), J EDEC MS-001, 0.300 Wide

Devices also available in Tape and Reel. Specify by appending the suffix letter "X" to the ordering code.

Connection Diagram



DM74LS283 4-BIT BINARY ADDER WITH FAST CARRY

74LS181 – 4-bit ARITHMETIC LOGIC UNIT

TABLE 2

SELECTION				ACTIVE-HIGH DATA		
				M = H LOGIC FUNCTIONS	M = L; ARITHMETIC OPERATIONS	
S3	S2	S1	S0		C _n = H (no carry)	C _n = L (with carry)
L	L	L	L	$F = \overline{A}$	$F = A$	$F = A \text{ PLUS } 1$
L	L	L	H	$F = \overline{A + B}$	$F = A + B$	$F = (A + B) \text{ PLUS } 1$
L	L	H	L	$F = \overline{AB}$	$F = A + \overline{B}$	$F = (A + \overline{B}) \text{ PLUS } 1$
L	L	H	H	$F = 0$	$F = \text{MINUS } 1 \text{ (2's COMPL)}$	$F = \text{ZERO}$
L	H	L	L	$F = \overline{AB}$	$F = A \text{ PLUS } \overline{AB}$	$F = A \text{ PLUS } \overline{AB} \text{ PLUS } 1$
L	H	L	H	$F = \overline{B}$	$F = (A + B) \text{ PLUS } \overline{AB}$	$F = (A + B) \text{ PLUS } \overline{AB} \text{ PLUS } 1$
L	H	H	L	$F = A \oplus B$	$F = A \text{ MINUS } B \text{ MINUS } 1$	$F = A \text{ MINUS } B$
L	H	H	H	$F = \overline{AB}$	$F = \overline{AB} \text{ MINUS } 1$	$F = \overline{AB}$
H	L	L	L	$F = \overline{A + B}$	$F = A \text{ PLUS } AB$	$F = A \text{ PLUS } AB \text{ PLUS } 1$
H	L	L	H	$F = A \oplus B$	$F = A \text{ PLUS } B$	$F = A \text{ PLUS } B \text{ PLUS } 1$
H	L	H	L	$F = B$	$F = (A + \overline{B}) \text{ PLUS } AB$	$F = (A + \overline{B}) \text{ PLUS } AB \text{ PLUS } 1$
H	L	H	H	$F = AB$	$F = AB \text{ MINUS } 1$	$F = AB$
H	H	L	L	$F = 1$	$F = A \text{ PLUS } A^\dagger$	$F = A \text{ PLUS } A \text{ PLUS } 1$
H	H	L	H	$F = A + \overline{B}$	$F = (A + B) \text{ PLUS } A$	$F = (A + B) \text{ PLUS } A \text{ PLUS } 1$
H	H	H	L	$F = A + B$	$F = (A + \overline{B}) \text{ PLUS } A$	$F = (A + \overline{B}) \text{ PLUS } A \text{ PLUS } 1$
H	H	H	H	$F = A$	$F = A \text{ MINUS } 1$	$F = A$

[†] Each bit is shifted to the next more significant position.

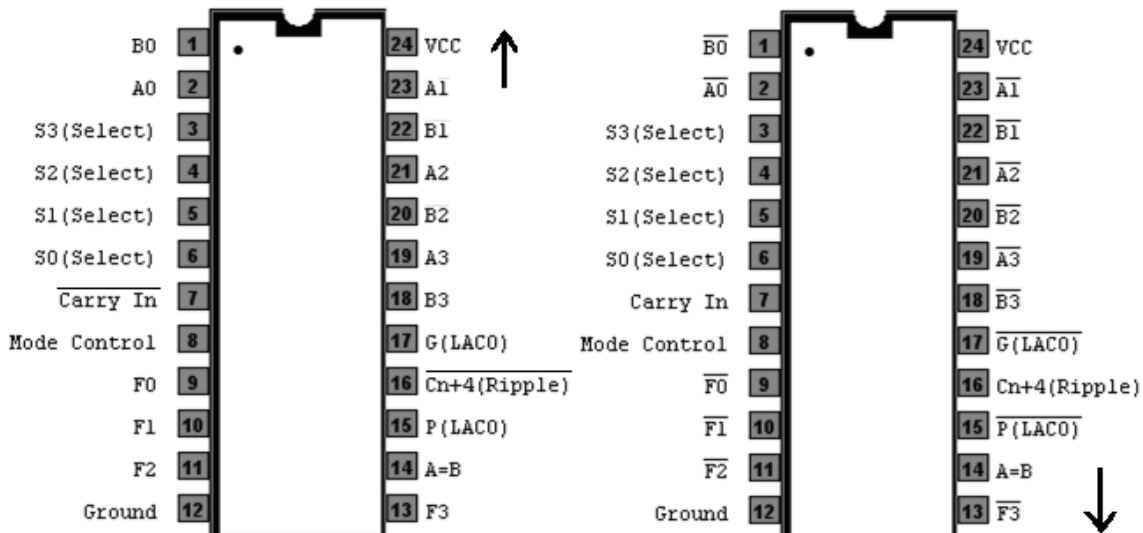


TABLE 1

SELECTION				ACTIVE-LOW DATA		
				M = H LOGIC FUNCTIONS	M = L; ARITHMETIC OPERATIONS	
S3	S2	S1	S0		C _n = L (no carry)	C _n = H (with carry)
L	L	L	L	$F = \overline{A}$	$F = A \text{ MINUS } 1$	$F = A$
L	L	L	H	$F = \overline{AB}$	$F = AB \text{ MINUS } 1$	$F = AB$
L	L	H	L	$F = \overline{A + B}$	$F = \overline{AB} \text{ MINUS } 1$	$F = \overline{AB}$
L	L	H	H	$F = 1$	$F = \text{MINUS } 1 \text{ (2's COMP)}$	$F = \text{ZERO}$
L	H	L	L	$F = \overline{A + B}$	$F = A \text{ PLUS } (A + \overline{B})$	$F = A \text{ PLUS } (A + \overline{B}) \text{ PLUS } 1$
L	H	L	H	$F = \overline{B}$	$F = AB \text{ PLUS } (A + \overline{B})$	$F = AB \text{ PLUS } (A + \overline{B}) \text{ PLUS } 1$
L	H	H	L	$F = A \oplus B$	$F = A \text{ MINUS } B \text{ MINUS } 1$	$F = A \text{ MINUS } B$
L	H	H	H	$F = A + \overline{B}$	$F = A + \overline{B}$	$F = (A + \overline{B}) \text{ PLUS } 1$
H	L	L	L	$F = \overline{AB}$	$F = A \text{ PLUS } (A + B)$	$F = A \text{ PLUS } (A + B) \text{ PLUS } 1$
H	L	L	H	$F = A \oplus B$	$F = A \text{ PLUS } B$	$F = A \text{ PLUS } B \text{ PLUS } 1$
H	L	H	L	$F = B$	$F = \overline{AB} \text{ PLUS } (A + B)$	$F = \overline{AB} \text{ PLUS } (A + B) \text{ PLUS } 1$
H	L	H	H	$F = A + B$	$F = (A + B)$	$F = (A + B) \text{ PLUS } 1$
H	H	L	L	$F = 0$	$F = A \text{ PLUS } A^\dagger$	$F = A \text{ PLUS } A \text{ PLUS } 1$
H	H	L	H	$F = \overline{AB}$	$F = AB \text{ PLUS } A$	$F = AB \text{ PLUS } A \text{ PLUS } 1$
H	H	H	L	$F = AB$	$F = \overline{AB} \text{ PLUS } A$	$F = \overline{AB} \text{ PLUS } A \text{ PLUS } 1$
H	H	H	H	$F = A$	$F = A$	$F = A \text{ PLUS } 1$

[†] Each bit is shifted to the next more significant position.